

Katedra informatiky
Přírodovědecká fakulta
Univerzita Palackého v Olomouci

BAKALÁŘSKÁ PRÁCE

Virtuální nástěnka pro podporu výuky na 1. stupni
základních škol



2025

Vedoucí práce:
RNDr. Martin Trnečka, Ph.D.

Michal Šmajzr

Studijní program: Informační technologie,
prezenční forma

Bibliografické údaje

Autor: Michal Šmajzr
Název práce: Virtuální nástěnka pro podporu výuky na 1. stupni základních škol
Typ práce: bakalářská práce
Pracoviště: Katedra informatiky, Přírodovědecká fakulta, Univerzita Palackého v Olomouci
Rok obhajoby: 2025
Studijní program: Informační technologie, prezenční forma
Vedoucí práce: RNDr. Martin Trnečka, Ph.D.
Počet stran: 32
Přílohy: elektronická data v úložišti katedry informatiky
Jazyk práce: český

Bibliographic info

Author: Michal Šmajzr
Title: Virtual bulletin board to support teaching in the 1st grade of primary schools
Thesis type: bachelor thesis
Department: Department of Computer Science, Faculty of Science, Palacký University Olomouc
Year of defense: 2025
Study program: Information Technologies, full-time form
Supervisor: RNDr. Martin Trnečka, Ph.D.
Page count: 32
Supplements: electronic data in the storage of department of computer science
Thesis language: Czech

Anotace

Tato práce se zabývá vytvořením webové aplikace, která se bude snadno používat učitelům na 1. stupni základních škol. Pro vytvoření webové aplikace jsem využil nejnovější technologie jako jsou framework Next.js, Tailwind a databáze MySQL, které mi umožnily vytvořit moderní online nástroj, který je do budoucna rozšiřitelný. Nástroj bude mít využití na základních školách, táborech, kroužcích nebo skupinových lekcích a umožňuje učiteli vytvářet nástěnky, sekce, příspěvky, ankety a posílat zprávy mezi učitelem a uživatelem.

Synopsis

This bachelor thesis deals with the creation of a web application that will be easy to use for teachers at the 1st level of primary schools. To create the web application, I used the latest technologies such as the Next.js framework, Tailwind and the MySQL database, which allowed me to create a modern online tool that is expandable in the future. The tool will be used in primary schools, camps, clubs or group lessons and allows the teacher to create bulletin boards, sections, posts, polls and send messages between the teacher and the user.

Klíčová slova: virtuální nástěnka; uživatelské rozhraní; REST API; Next.js; Tailwind; MySQL;

Keywords: virtual bulletin board; user interface; REST API; Next.js; Tailwind; MySQL;

Rád bych poděkoval svému vedoucímu práce RNDr. Martinu Trnečkovi, Ph.D. za jeho odborné rady.

Odevzdáním tohoto textu jeho autor/ka místopřísežně prohlašuje, že celou práci včetně příloh vypracoval/a samostatně a za použití pouze zdrojů citovaných v textu práce a uvedených v seznamu literatury.

Obsah

1	Výběr technologií	9
1.1	HTML	9
1.2	CSS	9
1.3	Tailwind CSS	9
1.4	JavaScript	9
1.5	TypeScript	9
1.6	React	9
1.7	Node.js	10
1.8	Next.js	10
1.9	MySQL	11
2	Architektura	11
2.1	Modulární architektura	11
2.2	REST API	12
3	Implementace	12
3.1	Návrh UI/UE	12
3.2	Návrh databáze	13
3.2.1	Transakce	14
3.2.2	Tabulky uživatelů	14
3.2.3	Tabulky nástěnek	14
3.2.4	Tabulky dotazů	16
3.2.5	Tabulky anket	16
3.2.6	Tabulky a záznamy pro odznaky (badges)	16
3.3	Autentizace	17
3.4	Implementace komponent	17
3.5	Implementace REST API	18
3.5.1	Struktura	18
3.5.2	Typování	18
3.5.3	Odpovědi	18
3.5.4	Odchycení chyb	19
3.5.5	Zabezpečení	19
3.5.6	Poskytnutí dat	19
3.6	Získávání dat	20
3.7	Posílání, mazání a změna dat	20
3.8	Nástěnky, sekce, příspěvky	20
3.8.1	Nástěnky a sekce	21
3.8.2	Příspěvky	21
3.8.3	Textové příspěvky	21
3.8.4	Fotografie	22
3.9	Komprese	22
3.10	Vyhledávání	22
3.11	Dotazy	23

3.12	Ankety	23
3.13	Správa uživatelů	24
3.13.1	Vytvoření uživatele	24
3.14	Nastavení uživatele	25
3.15	Mobilní verze	25
3.16	Bezpečnost	26
4	Porovnání s nástrojem Padlet	27
	Závěr	28
	Conclusions	29
A	První příloha	30
B	Druhá příloha	30
C	Obsah elektronických dat	30
	Literatura	32

Seznam obrázků

1	Stránka s nástěnky	13
2	EER diagram databáze	15
3	Mobilní verze	26

Úvod

V dnešní době existuje mnoho online nástrojů, které zastávají různé funkce, ale pro výuku převážně ve školství nejsou zcela vyhovující. Diskutoval jsem s učiteli a závěr byl jasný, dnešních online nástrojů je mnoho a učitelé i rodiče se v nich ztrácejí. Od školy dostanou nástroj pro nahrávání dokumentů, jsou nuceni komunikovat přes e-mail a používat externí nástroje na zpětnou vazbu. Proto se v této práci zabývám jak implementovat webovou aplikaci, který vše sjednotí a bude se snadno používat nejen učitelům, ale také rodičům.

V první kapitole pojednávám a seznamuji se základními technologiemi, které jsem použil a proč jsem je zvolil. Druhou kapitolu jsem věnoval architektuře webové aplikace. V třetí kapitole popisuji implementaci. Nakonec porovnávám svůj nástroj s existující alternativou jako je Padlet a v závěru prezentuji výsledky, kterých jsem dosáhl.

1 Výběr technologií

1.1 HTML

HTML (HyperText Markup Language) je hypertextový značkovací jazyk, který udává základní strukturu a sémantiku webu. Popisuje strukturu dokumentu a přes hypertextové odkazy propojuje jednotlivé dokumenty. Základním prvkem je element, který se skládá ze značek a může obsahovat atributy. [1]

1.2 CSS

CSS (Cascading Style Sheets), česky kaskádové styly, slouží k vizualizaci HTML elementů. [2] Jednotlivé styly bývá časem náročné udržovat, proto se využívají různé konvence a přístupy, jeden z nich je atomický design [3], každou vlastnost můžeme reprezentuje právě jednou třídou a díky tomu se udržuje nízká specifická a znovu použitelnost tříd.

1.3 Tailwind CSS

Jeden z nejznámějších a profesionálních frameworků, který implementuje atomický design [3], je utility first framework Tailwind CSS, kde právě každá jedna utilita reprezentuje jednu třídu. Všechny utility se zapisují přímo do HTML a není proto nutné mít mnoho souborů pro styly. Jeho aktuální nejnovější verzi 4 jsem použil pro psaní uživatelského rozhraní, protože umožňuje vysokou flexibilitu a udržitelnost. [4]

1.4 JavaScript

JavaScript je skriptovací jazyk, který slouží pro skriptování na straně klienta a umožňuje psát interaktivní webové stránky. Dnes se využívá jeho standard ECMAScript. [5]

1.5 TypeScript

JavaScript je dynamicky typovaný jazyk, což umožňuje měnit typ proměnných za běhu programu, ale zároveň to může způsobovat chyby. Proto jsem se rozhodl využít TypeScript, který do Javascriptu přidává statické typování a tím zvyšuje čitelnost kódu a snadněji se předchází chybám. [6]

1.6 React

React je knihovna, která zjednodušuje psaní uživatelského rozhraní. Umožňuje psát znovu použitelné komponenty, které mohou přijímat vlastnosti a udržují si svůj vnitřní čas. Funkční komponenty mohou používat React Hooks, což jsou funkce, které umožňují manipulovat s daty. [7]

1.7 Node.js

Node.js je framework, pomocí kterého lze psát plnohodnotný webový backend v JavaScriptu. Obsahuje moduly, kde každý modul zajišťuje určitou funkcionalitu. Další výhody jsou asynchronní zpracování a vysoká škálovatelnost. [8]

1.8 Next.js

Next.js je open source framework od Vercelu postavený na Reactu a umožňuje klientský, statický i server side rendering. Má v sobě zabudovaný Node.js server, proto je v něm možné psát plnohodnotný backend. Já jsem použil jeho aktuální nejnovější verzi 15, která přináší nový směrovač App Router. [9]

Mezi hlavními složkami a soubory bych vystihl následující:

- `public` – tato složka slouží pro uložení statických obrázků, které jsou následně dostupné přes URL, používám ji pro ikony, které jsem uložil do složky `icons`. Dále se zde nachází výchozí `favicon.ico`, která se zobrazuje v záložce webového prohlížeče a nahradil jsem ji vlastní ikonkou.
- `src` – při instalaci jsem zvolil možnost s vytvořením složky `src`, která mi může v budoucnu sloužit pro další rozšiřování aplikace, pokud se tato možnost nezvolí, vytvoří se zde přímo složka `app`.
- `app` – složka `app` slouží jako výchozí služba pro směrování v aplikaci. Každá vytvořená složka v `app` je dostupná přes URL a definuje danou trasu. Lze tedy vytvářet hierarchickou strukturu složek podle požadavků projektu, což je velmi přehledné a snadno se spravuje a rozšiřuje. V každé složce se nachází stránka `page.tsx` pro komponenty dané stránky a může se nacházet `layout.tsx`, kde lze definovat layout pro danou stránku a podstránky dané trasy. Také zde lze definovat dynamické trasy, které se vepisují do hranatých závorek, například `[userId]`. Jméno uvnitř závorek se používá pak k získání daného segmentu. Složka `api` v `app` lze strukturovat stejným stylem a umožňuje vytvářet API endpointy s názvem `route.ts`.
- `middleware.ts` – slouží pro provedení kódu ještě před dokončením požadavku. Používám pro zabezpečení přístupu ke stránkám.
- `.env (.env.local)` – slouží pro uložení konfiguračních dat. Mám zde definovaných několik proměnných jako `NEXTAUTH_SECRET` (tajný klíč k zabezpečení JWT tokenů), `NEXTAUTH_URL` (adresa k autentizaci) a `BASE_URL` (URL adresa stránky).

Dále jsem vytvořil několik složek pro další uspořádání projektu:

- `private` – `private` složku jsem vytvořil pro soubory a obrázky, které ukládám za běhu programu. Zpřístupňuji je přes zabezpečené API endpointy.

- `components` – do složky `components` ukládám jednotlivé komponenty. Pro jejich pojmenování jsem využil pascal case konvenci.
- `lib` – pro ukládání pomocných a konfiguračních souborů jsem vytvořil složku `lib`. Ukládám zde konfiguraci k databázi v souboru `db.ts`, pomocnou funkci knihovny `NextAuth.js` `auth()` v `auth.ts` a konfiguraci pro odesílání mailů v souboru `mailer.ts`.
- `types` – tuto složku jsem přidal do struktury, abych měl jednotné místo, kam ukládat rozhraní a typy v Typescriptu.
- `styles` – pro styly jsem vytvořil složku `styles`, kde ukládám soubor `globals.css`. Mám zde uloženou definici veškerých barev, které jsem vygeneroval dle Material design 3 pomocí Material Theme Builder. Také jsem zde dle stejné dokumentace definoval velikosti písma, zaoblení a stíny. Jako výchozí písmo jsem použil Roboto z Google fonts. Toto písmo mám definované jako proměnnou v `globals.css` a v layoutu `RootLayout` jsem písmo importoval a přidal konfiguraci jako jsou tloušťky (400, 500, 700), styly (normal, italic), znakovou sadu (latin-ext). Písmo nemusí být dostupné ihned, proto mám nastavený `display: "swap"`, který načte okamžitě záložní písmo dané výchozím prohlížečem a ve chvíli, co bude písmo stažené, se přepne.

Next.js disponuje dvěma základními principy vykreslování komponent. Prvním z nich je `server side rendering`, kdy komponenty vykreslí na serveru a uživateli pošle již hotovou stránku, v případě druhé možnosti se komponenty vykreslují na klientovi. V defaultním nastavení se všechny stránky vykreslují na serveru s výjimkou stránek, kde je uvedeno "use client", které se vykreslí na klientu. Výhoda `server side renderingu` je, že vše zpracuje server a klient dostane hotovou stránku, jistá nevýhoda je, že v server komponentách nelze použít `React hooks`, jako jsou například `useState`, `useEffect` a z toho důvodu spíše upřednostňuji klientské komponenty, abych mohl vytvářet interaktivní aplikaci.

1.9 MySQL

Z povahy dat bylo pro mě nejvýhodnější zvolit relační databázi, vybral jsem proto MySQL. K databázi je vždy dobré zvolit vhodné uživatelské rozhraní, které zjednodušuje její správu a vytváření dotazů, já jsem proto vybral z vlastní preference `phpmyadmin`, alternativně šel použít například `adminer`.

2 Architektura

2.1 Modulární architektura

Pro napsání webové aplikace v Next.js se mi nabízely dvě možnosti, první z nich byla použít monolitickou architekturu (všechno v jednom) a použít pro

psaní webového backendu Server Action (což jsou asynchronní operace, které běží v Node.js a lze je použít přímo v komponentách pro zpracování dat) nebo použít modulární architekturu s REST API. Rozhodl jsem se pro druhou možnost především kvůli rozšiřitelnosti samotné aplikace, jednotlivé API endpointy mi umožňují se neomezovat pouze na moji aplikaci, ale poskytovat je dále. Budu moci tedy v budoucnu aplikaci dále rozšiřovat a implementovat nativní mobilní či desktopovou aplikaci. Aplikace se tedy skládá z webového front-endu, REST API a databáze. Každý API endpoint je modul, který zajišťuje danou funkcionalitu, komunikuje s databází a vrací klientovi data.

2.2 REST API

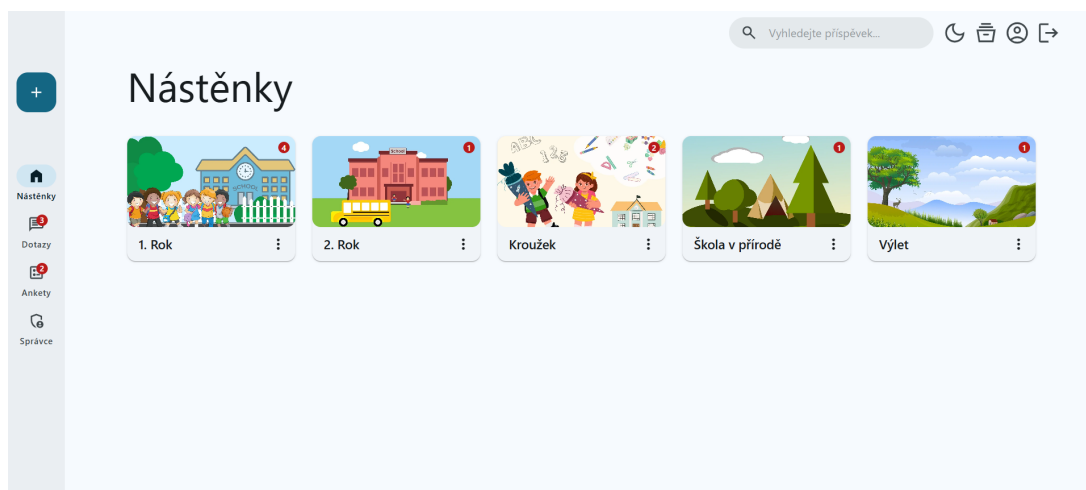
REST API, v překladu aplikační programové rozhraní založené na architektuře REST, bylo pro mě ideální řešení. Komunikace probíhá pomocí čtyř základních HTTP metod GET, POST, PUT, DELETE a každá z nich má svůj specifický účel: GET slouží pro získávání dat, POST pro jejich posílání, PUT pro změnu dat a DELETE slouží pro mazání dat. Pokrývají tedy všechny základní operace, které se provádí s daty nad databází.

Celé to tedy funguje následovně. Ve webové aplikaci se používá klasická klient server architektura. Klient, v našem případě webový prohlížeč, se dotazuje přes HTTP požadavek serveru, což jsou v mém případě API endpointy, které běží v Next.js (interně používá Node.js server) a ty obsluhují daný požadavek. API endpoint daný požadavek zpracuje, provede interakci s databází, od které získá data a tyto data pošle zpět v HTTP odpovědi klientovi, který je ve webovém prohlížeči vykreslí. [10]

3 Implementace

3.1 Návrh UI/UE

Před samotným napsáním webové aplikace je vždy důležité navrhnout uživatelské rozhraní. User interface se zabývá tím, jak webová stránka vypadá a user experience se zabývá rozložením prvku, aby se aplikace dobře používala, pro uživatelsky přívětivou stránku je nutné se zabývat oběma styly. Pro návrh se využívají designové systémy, které obsahují jednotlivé vzory, z kterých se bude uživatelské rozhraní skládat. Pro návrh komplexního designu je potřeba navrhnout převážně jednotlivé komponenty, barvy, typografii, ikony a layout (rozvržení). Já jsem se rozhodl použít Material Design 3 od Googlu, který obsahuje většinu těchto vzorů, z kterých jsem si mohl poskládat designový systém pro svoji webovou aplikaci. Material Design 3 jsem zvolil, protože se jedná o moderní design, kde jsou již všechny vzory odzkoušené pro co nejlepší uživatelskou přívětivost. Využil jsem tedy jejich návrh barev, typografii, ikony a návrh komponent, které jsem si sám implementoval v Reactu. Následně jsem si sestavil vlastní layout, který lze vidět na obrázku 1.



Obrázek 1: Stránka s nástěnky

Vlevo se nachází vedlejší navigační menu (komponenta Navigation rail), která slouží pro navigaci mezi hlavními stránkami a obsahuje komponentu Fab pro přidávání obsahu. Pro některé stránky jsem toto menu využil pro navigační šipku zpět, aby se uživatel mohl snadno přesunout na předchozí stránku. V horní části stránky se nachází navigační lišta pro základní operace a navigaci, jako jsou změna tmavého/světlého režimu, pro učitele archiv, uživatelská nastavení a odhlášení. Toto horní menu zobrazuji na všech stránkách, kde nezabírá místo pro hlavní obsah.

Hlavní obsah se nachází ve zbytku stránky. Většinou se skládá z velkého nadpisu, který vystihuje aktuální stránku a pod ním se již nachází samotný obsah. V případě stránky konverzace jsem se inspiroval designovým návrhem Material Design 3 kit, který se skládá z levého menu, kde je seznam konverzací a napravo se již nachází samotná konverzace. Podobný styl jsem zvolil i pro Ankety, kde jsem si design přizpůsobil vlastním potřebám.

3.2 Návrh databáze

Databáze je klíčová součást webové aplikace a slouží pro ukládání dat. Já jsem využil knihovnu mysql2, která slouží pro připojení a práci s databází MySQL v Node.js. Pro navázání spojení slouží soubor db.ts ve složce lib, v kterém vytvářím pomocí metody createPool pool s danými vlastnostmi. Host určuje IP adresu serveru a port, na kterém portu je spuštěna, user a heslo slouží k zadání přihlašovacích údajů a nakonec se uvede název databáze. Primární klíč každé tabulky jsem zvolil sloupec s názvem id a typem int, který je nastaven na auto increment, což znamená, že při přidání vždy zvýší hodnotu id o 1 a zaručuje tak unikátnost každého řádku. Pool funguje oproti connection následovně: Vytvoří se pool s počtem spojení (výchozích je 10, ale lze změnit ve vlastnostech), tyto spojení zůstávají otevřené a pokud potřebuji se připojit k databázi využiji právě

jedno spojení z poolu, poté co ho již nepotřebuji se automaticky vrátí do poolu. Tento přístup zvyšuje výkon při práci s databází, což je dobré právě při aplikaci, kterou využívá více uživatelů. Protože ve své aplikaci používám jednu aplikaci, tak mi stačí používat jeden pool, proto využívám návrhový vzor singleton, který vytvoří jeden sdílený pool a zamezí neustálému vytváření nových poolu, které můžou vést až k chybě Too many connection, kdy se vyčerpají veškerá spojení.

Pro samotné použití spojení v API stačí importovat dané spojení přes `import pool from "@lib/db"` a následně stačí již použít daný pool s danou metodou `execute` nebo `query`, dle případu užití. Následně se knihovna postará o získání spojení, provedení dotazu a vrácení spojení, což umožňuje promise verze knihovny s podporou asynchroních operací.

3.2.1 Transakce

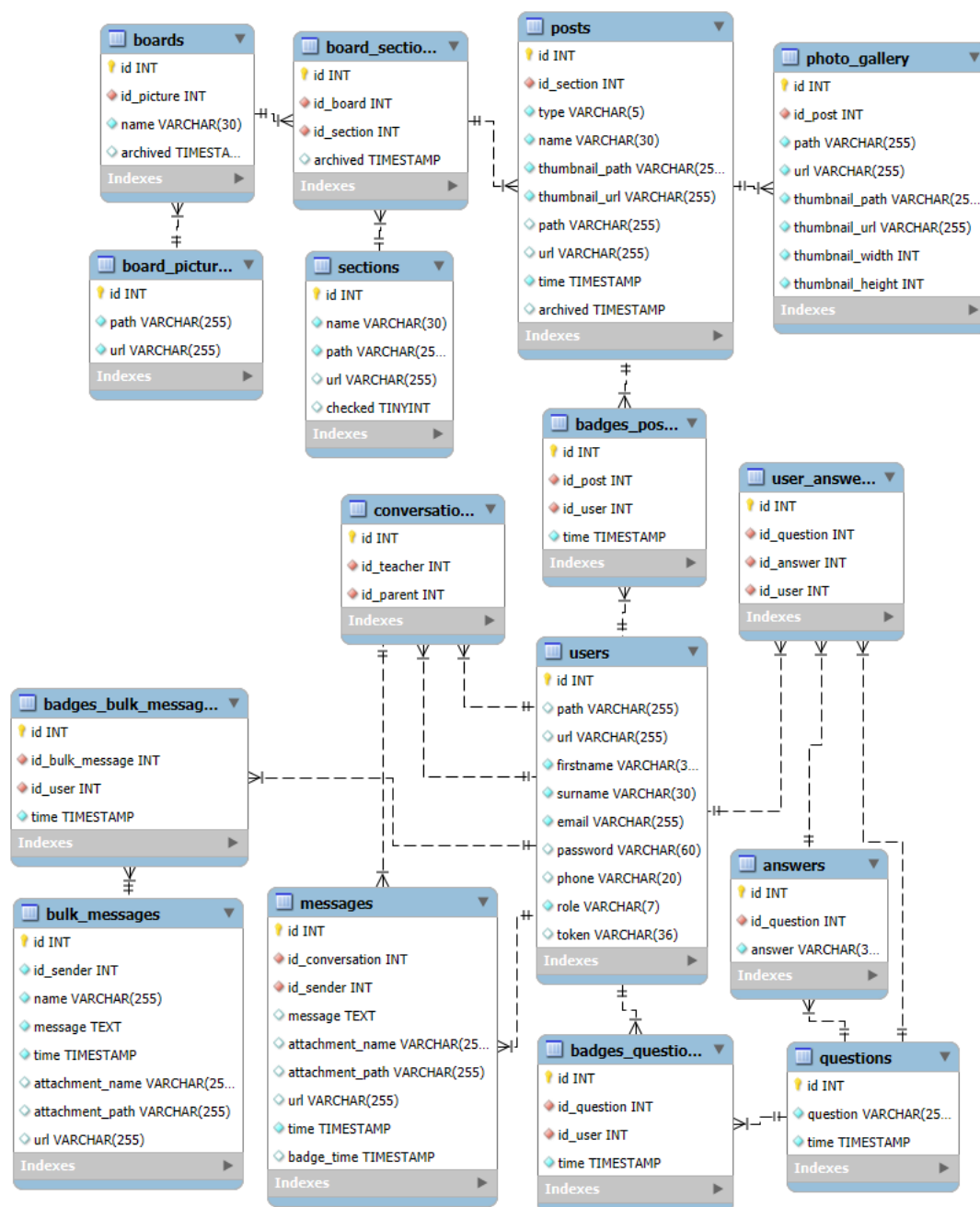
Pokud provádím zároveň více příkazů za sebou, může se stát, že by druhý příkaz selhal a první by se provedl a tím by se aplikace mohla dostat do nevalidního stavu. Tento problém řeším pomocí transakcí, kdy se dané dotazy v transakci zapíší až po potvrzení transakce. Transakce také používám v případech, kdy zapisuji na disk a mohlo by se stát, že zápis selže a nastal by stejný stav jako v prvním případě. Abych mohl implementovat transakce, tak z daného poolu musím získat spojení přes metodu `getConnection()`. Následně všechny dotazy provádím na daném spojení. Pro zahájení transakce slouží metoda `beginTransaction()` a po ní následují příkazy v dané transakci, které potvrzuji metodou `commit()`. V případě chyby vrátím změny přes `rollback()`. Na konci vždy vrátím dané spojení do poolu metodou `release()`.

3.2.2 Tabulky uživatelů

Všechny tabulky a propojení mezi nimi jsem zobrazil v EER diagramu na obrázku [2](#). Pro ukládání uživatelů jsem vytvořil tabulku `users`, která se skládá z křestního jména `firstname` a příjmení `surname`, které jsem nastavil na typ `VARCHAR(30)`, který dává dostatečný prostor i pro dlouhá jména. Tabulka obsahuje také sloupec `password`, kam ukládám hash hesla, který má vždy 60 znaků, proto jsem nastavil typ na `VARCHAR(60)`. V sloupci `role` ukládám jednu ze dvou rolí `teacher` a `user`, které reprezentují učitele a uživatele jako je třeba rodič, sloupec jsem natavil na typ `VARCHAR(7)` a je možné do něho v budoucnu ukládat více rolí. Další sloupce jsou pro telefonní číslo a `token`, který slouží pro ověření uživatele při nastavení hesla. Sloupec `path` určuje adresu k profilovému obrázku a sloupec `url` adresu `api`, přes které je profilový obrázek přístupný uživatelům aplikace.

3.2.3 Tabulky nástěnek

Tabulka `boards` slouží pro ukládání nástěnek, obsahuje název `name` a příznak `archived` zda je nástěnka archivována. Protože každá nástěnka může mít různý obrázek, vytvořil jsem tabulku `board_pictures`, která slouží pro uložení cesty



Obrázek 2: EER diagram databáze

k obrázku a cesty k api, přes které je obrázek dostupný. Tabulky jsou spojené přes cizí klíč na sloupci `id_picture`, který odkazuje na `id` v tabulce `id_pictures`. Podobně spojené jsou i tabulky `boards` a `board_sections`, kde ukládám aktuální sekce k nástěnce. Zde je trochu jiná struktura než u nástěnek, protože každá sekce má daný obrázek, proto ukládám název, `path`, `url` do jedné tabulky s názvem `sections`. V této tabulce je také sloupec `checked`, který slouží pro nastavení defaultních sekcí. Pokud je hodnota v tomto sloupci nastavená na 1, tak sloupec bude již předvybraný v seznamu při přidávání sekcí.

Příspěvky ukládám do tabulky `posts`, která obsahuje název `name`, cestu a `url` k náhledu, dále cestu a `url` k danému příspěvku, čas vytvoření `time` typu `timestamp` a příznak, zda je archivovaný. Jediná výjimka je u fotogalerie, kde z důvodu více dat na jeden příspěvek jsem vytvořil novou tabulku `photo_gallery`. V ní se nachází cizí klíč `id_post` na tabulku `posts` a dva další dva sloupce, které udávají šířku a výšku daného obrázku.

V tabulkách `boards`, `board_section` a `posts` indexuji sloupec `name` pro rychlejší a efektivnější vyhledávání.

3.2.4 Tabulky dotazů

Pro uchování konverzací slouží tabulka `conversation`, kde jsou dva sloupce `id_teacher` a `id_parent`, které reprezentují konverzaci mezi učitelem a rodičem a odkazují na tabulku `users`. Na tuto tabulku je navázána tabulka `messages`, kde jsou uloženy jednotlivé zprávy, každá zpráva může obsahovat přílohu `attachment`, kde ukládám její název, cestu a `url`. Dále v tabulce uchovávám `id_sender`, což je uživatel, který zprávu odeslal a čas `time`, kdy byla zpráva odeslána.

Další tabulka slouží pro hromadné zprávy, obsahuje `id` odesílatele, název zprávy, `message` zprávy, čas, kdy byla zpráva odeslána a přílohy, které mají stejnou strukturu jako v tabulce `messages`.

3.2.5 Tabulky anket

Pro ukládání otázek anket jsem vytvořil tabulku `questions`, kde je uložená daná otázka `question`, která může obsahovat až 255 znaků a aktuální čas, kdy byla otázka vytvořena.

Odpovědi ukládám do tabulky `answers`, kde je cizí klíč `id_question` na tabulku `questions` a odpověď, kterou jsem omezil na 30 znaků.

Pro uživatelské odpovědi jsem napsal tabulku `users_answers`, která obsahuje cizí klíče na tabulky `questions`, `answers` a `users`.

3.2.6 Tabulky a záznamy pro odznaky (badges)

Zbylé tabulky slouží pro odznaky, které slouží, aby uživateli sdělily, že přibyl nebo se změnil aktuální záznam. Když jsou data ve formátu 1:n (jeden záznam, který si může zobrazit více uživatelů), tak vytvářím nové tabulky, pro post to je tabulka `badges_posts`, pro hromadné zprávy `badges_bulk_messages` a pro

ankety tabulka `badges_question`. Pokud daný uživatel klikne na daný příspěvek, hromadnou zprávu nebo otázku, tak se do odpovídající tabulky vkládá záznam s jeho id, cizím klíčem na daný záznam a aktuálním časem. Pro zprávy, které jsou ve formátu 1:1 (mezi učitelem a rodičem) ukládám informaci o čase zobrazení přímo do tabulky `messages`.

3.3 Autentizace

Pro autentizaci jsem potřeboval spolehlivé a vyzkoušené řešení, které mi umožní si vytvořit vlastní přihlašovací formulář a poskytne mi základní logiku, vybral jsem proto knihovnu `NextAuth.js`.

Pro ověření používám `Credentials` s vlastním formulářem, kde po ověření uživatele přes uživatelské jméno (e-mail) a heslo uživatele přesměruji na dashboard. Pro podporu použití funkcí `signIn` a `signOut` mám v `auth.ts` v `pages` uvedenou trasu na vlastní formulář. V případě přístupu na zabezpečenou stránku bez ověření uživatele přesměruji přes `middleware`.

Pro ověřování jsem zvolil defaultní JWT strategii. JWT (JSON web token) je zašifrovaný token, který slouží pro ověření uživatele a ukládá se defaultně do zabezpečené session cookie. Funguje to tedy následovně – v souboru `auth.ts` mám definovanou funkci `authorize`, která slouží pro ověření uživatele, v této funkci volám API, která ověří uživatele s databází a následně pošle informaci o jeho id a roli, které se uloží do tokenu. Pro získání dat pro front-end používám session callback, který slouží pro uložení dat z tokenu do session, abych uživatele mohl ověřovat na front-endu. Následně mi stačí používat v komponentě `useSession`, z které získám status a session, ověřím, jestli status je `authorized` a jestli uživatel má danou roli. `NextOvěření` přes token pak používám v `middleware`, kde používám vestavěnou funkci `getToken`.

Pro zabezpečení API Routes jsem použil pomocnou funkci z dokumentace `auth`, která zpřístupňuje ověření uživatele přes `getSession`.

Do databáze ukládám hash se solí, který generuji s pomocí knihovny `bcrypt`.

3.4 Implementace komponent

Ve webových aplikacích se často nachází znovu se opakující prvky jako jsou například tlačítka, ovládací prvky, kartičky, navigace a layouty, abychom tyto prvky nemuseli neustále dokola implementovat, tak se nabízí jednoduché řešení vytvořit tento prvek pouze jednou a následně všude již používat jeho definici. Tento přístup výrazně zjednodušuje a zpřehledňuje kód a zároveň, pokud nás někdo požádá o změnu, tak nemusíme upravovat kód na někdy i desítkách míst, ale stačí nám jedna změna, která se provede všude, kde definici používáme. Tento problém řeší knihovna `React`, kterou jsem vybral právě pro implementaci těchto prvků, které nazýváme komponenty. V moderním `Reactu` se již upustilo od používání třídních komponent a v dnešní době se již používají funkční komponenty, jejichž syntaxe je mnohem jednodušší a přehlednější. Zároveň s nimi můžeme

používat React hooky, což jsou funkce, které umožňují udržovat a měnit stav funkční komponenty.

3.5 Implementace REST API

REST API je základní součástí mé webové aplikace a tvoří backend část aplikace. Pro implementaci jsem použil Next.js Route Handlers.

3.5.1 Struktura

API jsem strukturoval do několika tras podle účelu (každá složka reprezentuje danou trasu). Základní trasy jsou account, administration, archive, conversations, polls, které se dále větví. Dále mám API pro přihlášení na trasu login a trasu pro inicializaci rodiče init (tato trasa slouží pouze pro inicializaci a z bezpečnostních důvodů ji doporučuji po použití smazat). Pro některé badge jsem vytvořil zvlášť trasy, abych dodržel správnou strukturu. Následně jsem pro všechny trasy, které zprostředkovávají přístup k souborům ve složce private, vytvořil zvlášť trasy source, čímž jsem zjednodušil jejich správu.

V projektu používám dynamické segmenty trasy, které jsem vždy pojmenoval podle jejich účelu, abych určil jasný význam, k čemu slouží. K těmto segmentům přistupuji přes object params, který Next.js automaticky předává jako druhý argument vedle požadavku.

Z důvodu, že by uživatel mohl vkládat soubory se stejným názvem, tak pro každý soubor, který je nahrán přes API, vytvářím jedinečné uuid, které nastavím jako jeho nový název. Využívám k tomu JavaScript metodu randomUUID() z rozhraní Crypto.

3.5.2 Typování

V každém API endpointu nejdříve načtu přístup k databázi přes import pool from "@lib/db", kde pak přes objekt pool již volám metody query a execute. Načtu typy z knihovny mysql2, které následně používám, což jsou pro SELECT dotazy RowDataPacket a v případě dotazů jako jsou INSERT, UPDATE, DELETE ResultSetHeader.

3.5.3 Odpovědi

Odpovědi ve většině případů vracím ve formátu json, ve kterém vracím objekt v případě úspěchu s klíčem message, kde nastavím hodnotu například na success a v případě neúspěchu klíč error, kam umísťuji hodnotu podle dané chyby. Pro pojmenování hodnot jsem využil camel case konvenci. Dále vracím status kód, v případě úspěchu se automaticky vrací kód 200, a proto ho explicitně neuvádím. Přehled nejčastějších kódů s jejich názvy a vlastním popisem, které používám:

- 200 OK – úspěšný požadavek, většinou vracím message a data (pokud jsou požadována)

- 400 Bad Request – špatný požadavek, kdy klient poslal nevalidní data
- 401 Unauthorized – neautorizovaný uživatel, uživatel není přihlášen nebo nemá dostatečná práva
- 404 Not Found – nenalezen zdroj, pokud nelze najít požadovaný zdroj
- 500 Internal Server Error – chyba serveru, obecná odpověď při chybě na straně serveru

Následně na frontendu mohu přes tyto status kódy a error hodnoty rozlišovat chyby. Pro obecnější chyby používám status kódy, pokud chci podrobnější chybu, tak si přečtu hodnotu error a následně vypíšu odpovídající hlášku uživateli.

3.5.4 Odchycení chyb

Abych odchytil veškeré chyby, které mohou vzniknout, tak používám try a catch. Do try umístím kód, který chci kontrolovat, pokud by se vyskytla chyba, tak se vyvolá výjimka. Catch slouží k zachycení výjimky a dle paramtru v catch si mohu vypsát informace o dané chybě do konzole `console.error(error)`. Následně klientovi vrátím odpověď se status kódem 500 a univerzálním popisem v error. Mohu si definovat i vlastní chyby například `throw new Error("notFoundPhoto")` a následně chybu zpracovat v catch bloku. V některých případech potřebuji kód zavolat vždy na konci, k čemuž slouží block finally, který využívám například při transakcích.

3.5.5 Zabezpečení

Pro zabezpečení importuji pomocnou funkci `auth()` z knihovny `NextAuth.js`, přes kterou přistupuji ke `getSession`. Tuto funkci zavolám a získám danou session cookie z požadavku a zjistím, jestli je uživatel přihlášen a případně ověřím jeho roli.

3.5.6 Poskytnutí dat

Přes API, které slouží pro přímé poskytnutí dat (trasy `source`) nevracím data v JSON, ale přímo v těle HTTP odpovědi. Výhoda je, že k datům může přistupovat klient přímo přes daný API endpoint. Tato data mohou být jakýkoliv typ souborů. Pro posílání dat používám JavaScript konstruktor `Response()`. Jako první argument uvedu data, která chci poslat. Druhý argument je json objekt, v kterém mohu nastavit hlavičku HTTP odpovědi. Nastavuji například `Content-Length`, kde udávám velikost dat v těle. V každé hlavičce musím nastavit její typ `Content-Type`. K zjištění jakého typu jsou daná data používám knihovnu `mime`, která vrátí daný typ a tento typ následně vkládám do hlavičky. Z důvodu, že knihovna může vrátit null hodnotu v případě, že by daný typ nezjistila, tak tento případ ošetřuji a nastavuji daný typ na `application/octet-stream`, což značí

binární data s neznámým typem. V případě, že chci, aby daná data byla reprezentovaná jako příloha, tak nastavuji Content-Disposition na attachment a také zde nastavuji název daného souboru.

3.6 Získávání dat

Aby stránka byla plně interaktivní, tak používám React hooky jako jsou use-State a useEffect, které lze používat pouze v klientských komponentách. Z toho důvodu vytvářím v klientské komponentě asynchronní funkce začínající na load, které slouží k načtení dat. K získání dat slouží JavaScript funkce fetch, jejíž první argument je URL adresa, kam se posílá HTTP požadavek. Druhý argument je nepovinný, a když ho neuvedu, tak se automaticky použije metoda GET. Případně mohu uvést objekt, kde mohu nastavit danou metodu a hlavičku. Používám také klíčové slovo await, aby se počkalo na odpověď z požadavku. Následně odpověď zpracovávám. Pokud vše proběhlo v pořádku a byl vrácen status 200 (ok), tak daná data uložím do stavu, pokud ne, vypíšu chybovou hlášku. Celou tuto funkci vložím do useEffect, kde mohu do dependency pole (pole hodnot proměnných) vložit závislosti. V případě, že data načítám pouze při prvním renderu, tak se uvádí pole prázdné.

3.7 Posílání, mazání a změna dat

Kromě získávání dat mohu pomocí fetch funkce také data posílat. Nejdříve uvedu URL API endpointu, kam data posílám, a v druhém argumentu objekt, v kterém nastavím metodu, kterou budu používat. Pro posílání dat slouží POST, pro změnu dat PUT a pro mazání DELETE. V případě metody POST a PUT mohu posílat data a vkládat je do body HTTP požadavku. V případě, že data jsou objekt JSON, musím použít metodu JSON.stringify() a hlavičku nastavit na "Content-Type": "application/json".

3.8 Nástěnky, sekce, příspěvky

Pro implementaci informační nástěnky jsem zvolil hierarchickou strukturu podobnou složkám, což zajišťuje přehlednost a jednoduché používání. První úroveň jsou nástěnky a druhá úroveň jsou sekce, do kterých vkládám příspěvky. Tento přístup zajišťuje učitelům velkou flexibilitu s možností vše uspořádat dle svého uvážení.

Nejdříve jsem vytvořil komponentu Card (karta), jejíž základní funkcí je zobrazit obrázek, jméno a přesměrovat uživatele na další stránku. Učitelé navíc umožňují rozkliknout menu (tlačítko s třemi tečkami), kde může přesunout danou kartu do archivu. Pro navigaci jsem vytvořil komponentu Navigation, která slouží pro přesměrování a zároveň zobrazuje aktuální cestu mezi nástěnkami a sekcemi.

V případě učitele se v levém menu nachází komponenta Fab, což je tlačítko s obrázkem plus, která přesměruje učitele na průvodce vytvoření. V případě více kroků v horním okraji zobrazují progress bar, který učitelům procentuálně

zobrazuje, v jakém kroku se nachází. Vždy je nutné nejdříve určit název, který se vepisuje do komponenty `TextField`. Vstup musí být nenulový a maximálně 30 znaků, vstup kontroluji nejdříve na frontendu a z důvodu bezpečnosti i na backendu. Pokud uživatel zadá chybný vstup, tak nemá možnost pokračovat na další krok, ke kterému slouží tlačítko Další a vypíše se pod vstupním polem chybová hláška. Krokovač jsem implementoval přes stav, kde si ukládám aktuální číslo kroku a podle něho zobrazuji a kontroluji aktuální obsah.

3.8.1 Nástěnky a sekce

V případě nástěnky jsou celkem tři kroky, v prvním se nastaví název, v druhém obrázek a v třetím se ze seznamu vybere sekce. V druhém kroku je možné přidat vlastní obrázek, který se přidá do výběru obrázků, které lze použít pro nástěnky. V třetím kroku lze vybrat sekce a také vytvořit novou sekci s novým jménem a obrázkem, který se přidá do výběru sekcí. Zvolil jsem přístup, že každá sekce má daný obrázek, aby si uživatel vždy daný obrázek spojil s jejím účelem, což zvyšuje uživatelskou přívětivost. V posledním kroku se při kliknutí na tlačítko Vytvořit odešlou všechna data na API, kde je zpracuji a vrátím v případě chyby chybovou hlášku, nebo nástěnky vytvořím a přesměruji učitele na hlavní stránku Nástěnky.

V případě přidání sekcí se zobrazí stránka se sekcemi, které lze do dané nástěnky přidat a kde lze také přidat novou sekci do výběru.

3.8.2 Příspěvky

V aplikaci lze přidávat šest druhů příspěvků. Při přidání učitel každému příspěvku v průvodci nastaví jméno a následně v případě souboru, PDF, videa a audia nahraje dané médium a při kliknutí na Vytvořit se odešle na API, kde data od uživatele zpracuji, uložím na disk, do databáze a příspěvek zobrazím. Při vytvoření příspěvku vytvářím na disku hierarchickou strukturu s aktuální cestou k danému příspěvku, cesta začíná složkou boards, následuje id boards, section a posts, v kterém se již nachází daný příspěvek. Tento přístup mi umožňuje použít rekursivní mazání.

3.8.3 Textové příspěvky

Pro implementaci textových příspěvků jsem zvolil `TipTap`. Editor je headless, což znamená, že mi poskytuje logiku a zároveň jsem si mohl vzhled vytvořit sám a tím dodržet design mé aplikace. Po vytvoření či uložení příspěvku ukládám data ve formátu json, což je jeden z formátů, který editor podporuje. Kdykoliv chci textový dokument zobrazit, nahraji tento json do editoru a zobrazím ho. Neoprávněním uživatelům místo menu s úpravami zobrazím název dokumentu a schovám tlačítko pro uložení.

3.8.4 Fotografie

Pro fotografie jsem vytvořil fotogalerii, kam lze nahrát jeden a více obrázků. Z důvodu optimalizace jsem omezil jednu fotogalerii na 100 fotek a v případě potřeby mohu počet fotek navýšit. Aby si uživatel mohl prohlédnout náhledy a případně upravit fotky ještě před posláním, tak jsem využil knihovnu `react-images-uploading`, která zajišťuje základní logiku a zobrazení fotek na klientovi a následně jsem si vytvořil vlastní design pro jejich úpravu. Po nahrání fotek je přes form data odešlu na backend, kde je dále zpracovávám. Základní informa o názvu a miniatuře ukládám do tabulky `posts`, následně informace o dané fotce a jejím náhledu do tabulky `photo_gallery`. Na disku ukládám jednotlivé komprimované fotky a jejich náhledy do vytvořené složky `thumbnails`, miniaturu pro daný příspěvek ukládám do složky `thumbnail`. Pro zobrazení fotogalerie jsem využil knihovnu `yet-another-react-lightbox` s knihovnou `react-photo-album`, která slouží pro zobrazení náhledů.

3.9 Komprese

Veškeré obrázky, které uživatel nahraje prochází kompresí, kterou dělám z důvodu optimalizace, protože uživatel může nahrát moc velké obrázky, které by se následně dlouho načítali a zabírali zbytečně moc místa na disku. K tomu využívám knihovnu `sharp`, jež disponuje funkcí `sharp()`, která přijímá jako první argument buffer typu `Buffer`, kde je nahrán daný obrázek. Následně používám metodu `resize()`, která zmenší obrázek na rozměry, které se uvedou jako argument, následně používám metodu `toFile()`, která jako argument přijímá cestu k souboru. Obrázky prochází nejen kompresí, ale také měním jejich velikost. Pro ikony jsem zvolil šířku 320 px, pro miniatury a náhledy 1280 px a pro obrázky ve fotogalerii 1920 px.

3.10 Vyhledávání

Vyhledávání je jednou z nejčastějších funkcí, kterou lze najít ve webových aplikacích. Vyhledávat jsem umožnil v příspěvcích, v sekcích i nástěnkách, což uživateli zrychluje přístup k daným příspěvkům, které zrovna hledá.

Pro vyhledávání jsem vytvořil komponentu `Search` a pro její implementaci jsem využil návod z oficiálních stránek `Next.js`. Komponenta přijímá dva argumenty `placeholder` a `isOpen`, kde `placeholder` je textový řetězec, který se zobrazí, pokud je vstupní pole prázdné, `isOpen` je stav, zda se má vyhledávání zobrazit, což je z důvodu mobilní verze, kdy vyhledávání zobrazuji při kliknutí na ikonu vyhledávání. Nejdříve uživatel zapíše vstup přes `input`, kterému jsem nastavil typ `search`, tento vstup jde jako argument funkce `handleSearch`, která vstup dále zpracovává. Následně používám parametry vyhledávání, to znamená, že se vstup vepíše do url, z které následně vstup zpracovávám. Výhoda tohoto přístupu je, že si uživatel může danou adresu pro vyhledání zkopírovat do záložek a v

budoucí ji kdykoliv načíst. Pro přístup k parametru z url slouží funkce `useSearchParams` a pro její manipulaci se používá `API URLSearchParams`. Například z url ve tvaru `?query=video` (které vrátí hook `useSearchParams`) `URLSearchParams` získá parametr `video`. Následně tento parametr můžu přes metodu `set` nastavit na vstup od uživatele a nebo případně parametr metodou `delete` z URL smazat. K propsání změn do URL se následně používá metoda `replace` z `Next.js` hooku `useRouter`. Aby se hodnota z url při prvním načtení projevila ve vstupu, tak je potřeba ještě nastavit `defaultValue` na aktuální parametr z URL. [11]

3.11 Dotazy

Aby se mohl uživatel (rodič) dotázat učitele, tak při vytvoření uživatele vždy vytvářím novou konverzaci mezi uživatelem a rodičem. Tato konverzace se následně zobrazí na stránce `Dotazy`, kde se nachází v levém sloupci seznam konverzací a v pravém sloupci se zobrazuje aktuální konverzace. Tento přístup nahrazuje klasickou potřebu vytvářet neustále nové konverzace nebo e-maily a zjednodušuje a zphřehledňuje rodiči práci. K napsání zprávy slouží vstupní pole a k odeslání tlačítko napravo. Jako zprávu nebo se zprávou lze odeslat přílohu, ke které slouží tlačítko nalevo s ikonou přílohy. Následně se zprávy vždy řadí od poslední odeslané k té nejstarší, aby měl uživatel vždy přístup k té aktuální. Pro obnovování zprávy a konverzací mám nastavený v `useEffect` interval na 10 sekund. Pokud by byl požadavek na co nejkratší odezvu, tak bych mohl implementovat `WebSocket`, ale vzhledem k počtu uživatelů a frekvencí dotazů není potřeba. Jednotlivé konverzace a zprávy ukládám do stavů, které při odeslání posílám na backend, kde je následně zpracuji.

Další užitečnou funkcí jsou hromadné zprávy, které může učitel poslat všem uživatelům aplikace. I k těmto hromadným zprávám lze přiložit přílohu a také název, který může být dlouhý až 255 znaků.

Pro zprávy i hromadné zprávy jsem se rozhodl uživatele neomezovat a použil typ `text`, který umožňuje uložit až 65 535 bytů na jednu zprávu. Případně bych mohl nastavit limit přes typ `VARCHAR` a tím uživateli omezit počet znaků, které může ve zprávě poslat.

3.12 Ankety

Aby učitel mohl získat rychlou zpětnou vazbu od uživatelů, tak jsem vytvořil ankety. Při vytvoření anket rodič zadá otázku, která může být dlouhá 255 znaků a odpověď, kterou jsem nastavil na 30 znaků. Odpovědi může být dvě a více, aby měl rodič alespoň dvě možnosti, z kterých může vybrat. Po vytvoření ankety učitel vidí název a u každé odpovědi počet hlasů. Pokud klikne na tlačítko `Zobrazit výsledky`, tak se mu ukáže přehledné okno, v kterém vidí nalevo jednotlivé odpovědi a napravo, kdo pro danou odpověď hlasoval, může si tak přehledně prohlížet, jak hlasovali jednotliví uživatelé. Uživatel může pro odpovědi hlasovat, když hlasuje poprvé, tak se záznam vkládá a následně, pokud uživatel změní odpověď,

tak v databázi hodnotu aktualizují.

3.13 Správa uživatelů

Stránka se správou uživatelů se zobrazuje pouze učitelům. Pro správu uživatelů jsem zvolil tabulku, kde v každém záznamu zobrazuji informace o uživateli. Učitel si tak může jednoduše dohledat informaci o rodiči. Po kliknutí na záznam může zobrazit velký náhled o daném uživateli. Rodiči umožňuji dále měnit informace o rodiči, jako je jméno, příjmení, telefonní číslo. e-mailová adresa je spjatá s účtem, a proto tuto informaci měnit neumožňuji. Dále rodič může resetovat heslo, čímž odešle uživateli e-mail s odkazem na nastavení nového hesla. Dále může rodič uživatele smazat, čímž se smažou i veškeré konverzace s daným uživatelem, proto před těmito akcemi používám dialogová okna, aby se náhodou nestalo, že by se překliknul.

3.13.1 Vytvoření uživatele

Při vytvoření uživatele validuji jednotlivé vstupy. Abych ověřil, zda se jedná o validní e-mail, využívám knihovnu validator. Tato knihovna disponuje objektem validator a metodou isEmail, která přijímá jako argument danou e-mailovou adresu. Po ověření mi vrátí logickou hodnotu, dle které mohu v případě nevalidní hodnoty zobrazit chybovou hlášku. Pokud je e-mail validní, tak ho s dalšími daty pošlu na API, kde data dále zpracovávám. Aby bylo nastavení hesla pro uživatele transparentní a bezpečné, zvolil jsem přístup, kdy uživateli posílám e-mail s odkazem na stránku s přihlášením. Pro odesílání e-mailu jsem využil knihovnu nodemailer a pro její nastavení jsem vytvořil funkci v souboru mailer.ts ve složce lib. V této funkci vytvářím uuid, které bude sloužit jako token pro ověřování, následně vytvořím link s adresou na moji stránku a jako parametr ?token= uvedu právě daný token. Tento link bude sloužit uživateli k přihlášení a při přihlášení budu token ověřovat s databází, kam ho při vytvoření uživatele uložím. Následuje samotná funkce pro nastavení adresy a portu SMTP serveru. Nastavuji hodnotu secure pro nezašifrované spojení false a pro zašifrované true. Pro testování odesílání e-mailu jsem zvolil službu Ethereal, která používá nezašifrované spojení, v případě nasazení by se použila jiná služba, která by měla být vždy zašifrovaná. Dále nastavuji přihlašovací údaje k SMTP serveru. Nakonec pošlu přes asynchronní funkci e-mail s danými údaji a odkazem. Po rozkliknutí odkazu se uživatel přesměruje na stránku pro nastavení hesla, kde heslo musí uvést dvakrát, aby se nestalo, že by se například překliknul. Po odeslání ověřím z parametru token s databází a uživatele přesměruji na stránku s přihlášením a v případě chyby vypíšu danou chybovou hlášku.

Dále ověřuji a formátuji zadané telefonní číslo. Pro ověření jsem použil knihovnu libphonenumber-js a nastavil jako defaultní české telefonní číslo. Knihovna tedy automaticky přes funkci parsePhoneNumber číslo naformátuje a přidá českou předvolbu, v případě jiné než výchozí předvolby se před telefonní číslo uvede daná předvolba a knihovna číslo naformátuje podle pravidel dané země.

3.14 Nastavení uživatele

Nastavení uživatele je stránka, která je přístupná přes horní menu, v případě mobilní verze přes hamburger menu. Jak název napovídá, stránka slouží pro uživatelská nastavení. Umožňuji zde uživateli změnit obrázek. Vytvořil jsem proto dialogové okno, kde lze daný obrázek nastavit a při nahrání obrázku zobrazuji jeho náhled, který ukládám do dočasné URL adresy na klientovi přes objekt v JavaScriptu `URL.createObjectURL()`. Po nahrání profilové fotky se zobrazí aktuální obrázek. Z důvodu, že používám API, které zpřístupňuje danému uživateli obrázek, tak aby webový prohlížeč zobrazil okamžitě aktuální obrázek a nenahrával obrázek, která má uložený v cache paměti, tak po získání dat přidávám k dané URL parametr `uuid` s jedinečným identifikátorem.

Dále na stránce umožňuji změnit uživatelská data jméno, příjmení a telefonní číslo, které formátuji stejně jak v případě správě uživatelů přes knihovnu `libphonenumber-js`.

V poslední části stránky si může uživatel změnit heslo, nejdříve je nutné zadat původní heslo a následně nové heslo, které musí potvrdit. V případě zadání špatného původního hesla, nebo pokud se nová hesla neshodují, uživateli vypíšu chybovou hlášku.

3.15 Mobilní verze

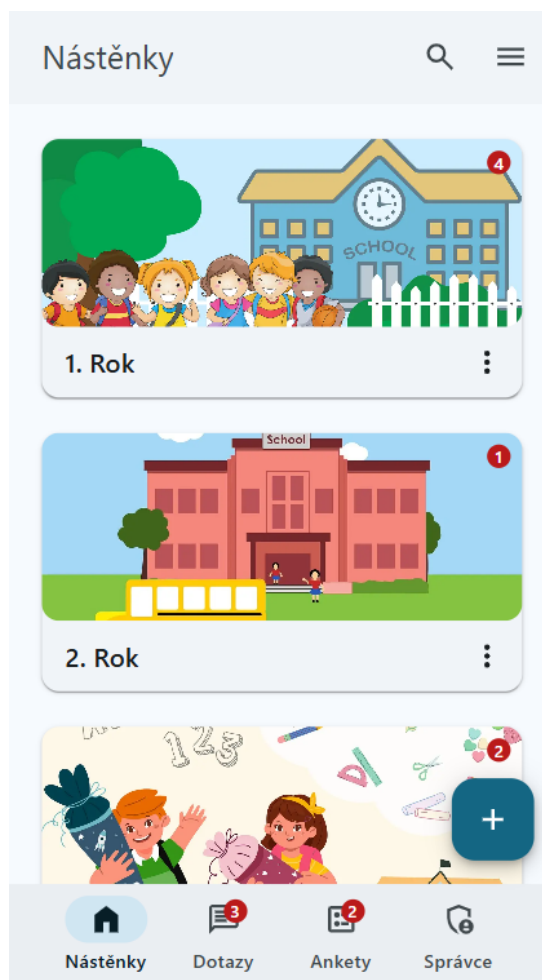
Dnes již mnoho uživatelů používá webové aplikace na mobilních zařízeních a tabletech, a proto jsem implementoval plně responzivní aplikaci, kterou vidíte na obrázku 3. Aplikace se automaticky přizpůsobuje šířce obrazovky.

Responzivní webovou stránku jsem implementoval převážně přes Tailwind, kde se vždy před danou utilitu uvedou prefixy, které fungují jako breakpointy media queries. Nejdříve se tedy definují všechny utility pro mobilní verzi a následně se podle velikosti obrazovky udávají prefixy jako jsou `sm:`, `md:`, `lg:`, `xl:`, `2xl`. Tomuto přístupu se říká *mobile first*, což má výhody především v rychlejší načítání na mobilních zařízeních.

Zároveň, pokud nechci nějakou část zobrazit na mobilní zařízení, tak používám utilitu `hidden`, která nastaví `display: none`; a následně zviditelním přes nastavení `display` vlastnosti přes utility jako jsou nejčastěji `block`, `inline`, `inline-block`, `grid` a `flex`.

Pro podmíněné vykreslování jsem zvolil knihovnu `react-responsive`, která umožňuje vytvořit proměnné dle breakpointů. Můžu díky tomu například ve funkcích určit, co se má vykreslit pouze pro počítačovou verzi.

Pro mobilní zařízení jsem vytvořil nové hamburger menu, které se nachází vpravo nahoře. Po rozkliknutí se objeví pravé menu, v kterém se nachází navigační prvky jako jsou uživatelské nastavení, pro učitele archiv, změna tmavého/světlého režimu a odhlášení, které lze nalézt v horním menu v počítačové verzi. Horní menu jsem využil pro zobrazení nadpisu dané stránky, na některých stránkách zobrazuji také šipku zpět. Také jsem přidal ikonu pro vyhledávání v odpovídajících stránkách. Levé menu jsem přesunul na spodní část obrazovky a



Obrázek 3: Mobilní verze

tlačítko Fab jsem umístil pravo dole nad ním a zpřístupnil tak veškeré ovládací prvky. U sekce dotazy a ankety jsem vše přizpůsobil pro co nejsnadnější používání, proto pravou část zobrazuji až po kliknutí, a poté schovávám spodní menu a Fab, které nejsou v daný moment potřeba, zpět na hlavní stránku se lze dostat přes šipku zpět v horním menu. V správě uživatelů vždy omezuji počet sloupců v tabulce podle šířky obrazovky, díky tomu je vždy vidět celá tabulka, v případě zobrazení všech informací lze přejít do náhledu uživatele.

3.16 Bezpečnost

Bezpečnost webových aplikací je naprosto zásadní. Z důvodu, že jsou dostupné komukoliv v síti, je nutné aplikaci zabezpečit. Já jsem proto zvolil několik osvědčených přístupů pro zabezpečení mé aplikace.

Pro zamezení přístupu na stránky, kam uživatel nemá právo vstoupit, pokud není například přihlášen nebo nemá odpovídající roli, jsem využil middleware, který umožňuje spustit kód ještě před dokončením požadavku. Všechna pravidla

jsem uvedl k tomu určenému souboru `middleware.ts`, kde nejdříve získám token přes funkci `getToken` (funkce z knihovny `NextAuth.js`), z které zjistím, jestli existuje ověřovací token pro daného uživatele, tedy zda je uživatel přihlášen, pokud není, tak ho přesměruji na stránku s přihlášením. Dále zjišťuji z tokenu, zda daný uživatel není učitel (role `teacher`) a pokud není, tak porovnám, zda chce přistoupit na jednu ze stránek, které jsou vyhrazeny pouze pro učitele, pokud by chtěl přistoupit na jednu z tras, kam nemá přístup, přesměruji ho na stránku s nástěnkami.

Jedním z častých útoků je SQL injection, kdy útočník chce přes nechráněný vstup dělat neoprávněné změny v databázi. Z toho důvodu jsem se rozhodl pro parametrizované dotazy, které umožňuje knihovna `mysql2`. To znamená, že pokud potřebuji do dotazu vložit nějakou proměnnou, nekládám ji přímo, ale vytvářím nejdříve pole, kam uložím veškeré proměnné a následně v samotném příkazu využiji zástupný symbol `?` za každou proměnnou. Následně je nutné použít metodu `execute` a té předám jako první parametr sql příkaz a jako druhý parametr pole s hodnotami a metoda se postará o bezpečné provedení příkazu.

Dalším problémem by mohlo být, kdyby uživatel mohl vkládat nulový nebo neplatný vstup a zároveň když překročí velikost vstupního pole, což by v aplikaci nebylo žádoucí a zároveň mohlo vést k různým chybovým stavům. Proto v API kontroluji, zda nedostávám od uživatele neplatné hodnoty nebo zda nepřekročil limit znaků pro daný vstup.

4 Porovnání s nástrojem Padlet

Padlet je nástroj, který umožňuje přidávat do nástěnky různé příspěvky a nasdílet ji přes odkaz uživatelům. Její omezení je především v jejím účelu, kdy slouží pro živou spolupráci mezi několika uživateli. Proto není úplně vhodná jako informační nástěnka pro trvalé uložení dat. Já jsem se proto snažil jít jinou cestou a vytvořit plnohodnotnou informační nástěnku s hierarchickou strukturou. V Padletu také není možnost spravovat dané uživatele, lze jim nasdílet nástěnku, ale již není možné mít seznam uživatelů se základními údaji, jako je třeba telefonní číslo, které často učitel potřebuje velmi rychle, proto tyto informace zobrazuji v přehledné správě uživatelů. V Padletu je sice možné vytvořit provizorní anketu v nástěnce, kde se na ní mohou dotazovat, ale již neexistuje přehledné jedno místo, kde jsou dostupné všechny ankety. Sice v Padletu lze okomentovat danou nástěnku, ale již není možná přímá komunikace mezi uživateli.

Padlet je jeden z mnoha nástrojů, které mají jistý účel, ale nejsou vhodné jako plnohodnotná informační nástěnka právě z důvodu jejich zaměření. Těchto nástrojů je mnoho, ale pro učitele to znamená používat velké množství takových aplikací a velice rychle se v nich ztrácejí jak samotní učitelé, tak i rodiče. Proto jsem vytvořil webovou aplikaci, která tento problém řeší a poskytuje učitelům komplexní nástroj pro jejich výuku.

Závěr

V bakalářské práci jsem popsal komplexní řešení webové aplikace, která bude sloužit učitelům pro jejich výuku. Vytvořil jsem přehledného průvodce pro vytváření nástěnek a výběru sekcí. Umožnil jsem přidání několika druhů příspěvků, které učitel běžně potřebuje a přidal jsem vyhledávání jak v příspěvcích, tak v sekcích a nástěnkách. Vytvořil jsem přehlednou správu uživatelů, která nejen poslouží účelům administrace, ale také jako informační tabulka pro základní informace, které učitel potřebuje o rodičích. Pro dotazy jsem implementoval konverzace, kde lze kromě zpráv posílat i přílohy. Dále jsem učiteli umožnil poslat hromadné zprávy pro všechny uživatele a vytvářet ankety, kde se může dotazovat. Vše jsem vytvořil v moderním a plně responzivním designu, který se bude snad uživatelům dobře používat. Díky její struktuře a použití nových technologií lze nástroj dále vyvíjet a rozšiřovat o další funkce.

Conclusions

In my bachelor's thesis, I described a comprehensive solution for a web application that will serve teachers for their teaching. I created a clear guide for creating bulletin boards and selecting sections. I enabled adding several types of posts that teachers commonly need, and I added search in posts, sections, and bulletin boards. I created a clear user management that will not only be used administration purposes, but also as an information table for basic information that teachers need about parents. For questions, I implemented conversations that allows sending not only messages but also attachments. I also enabled teachers to send bulk messages to all users and create polls where they can ask questions. I created everything in a modern and fully responsive design that users will use well. Thanks to its structure and the use of new technologies, the tool can be further developed and expanded with additional functions.

A První příloha

Text první přílohy

B Druhá příloha

Text druhé přílohy

C Obsah elektronických dat

Na samotném konci textu práce je uveden stručný popis obsahu elektronických dat odevzdaných v systému katedry informatiky spolu s textem. Tato data jsou nedílnou součástí práce a tvoří (datovou) přílohu textu práce. Povinné položky struktury dat jsou:

text/

Adresář s textem práce ve formátu PDF, vytvořený s použitím závazného stylu KI PřF UP v Olomouci pro závěrečné práce, včetně všech (textových) příloh, a všechny soubory potřebné pro bezproblémové vytvoření PDF dokumentu textu (případně v ZIP archivu), tj. zdrojový text textu a příloh, vložené obrázky, apod.

README.*

Textový soubor (s příponou např. `.txt`) s informacemi o opakovatelném způsobu použití ostatních dat práce – typicky plně reprodukovatelný co nejúplnější funkční postup zprovoznění software vytvořeného v rámci práce, tzn. jeho případné instalace/nasazení a spuštění, včetně uvedení všech požadavků pro bezproblémový provoz; za zprovoznění software se nepovažuje zpřístupnění (např. po Internetu) již někde zprovozněného software.

Adresáře a soubory s veškerými ostatními autorskými daty práce (případně v ZIP archivu) – typicky spustitelné a další soubory software vytvořeného v rámci práce potřebné pro bezproblémový provoz software, případně jeho instalační program, a kompletní zdrojové texty software a další data nutná pro plně reprodukovatelné korektní vytvoření spustitelných souborů.

Dále mohou data obsahovat například:

- ukázková a testovací data použitá v práci nebo pro potřeby posouzení práce v rámci její obhajoby,
- položky bibliografie v elektronické podobě, příp. jiná relevantní literatura a dokumentace vztahující se k práci,

- cizí data (software) potřebná pro bezproblémové použití autorských dat práce (software), která nejsou standardní součástí předpokládaného (softwareového) vybavení uživatele.

U veškerých cizích obsažených materiálů jejich zahrnutí dovolují podmínky pro jejich veřejné šíření nebo přiložený souhlas držitele práv k užití. Pro všechny použité (a citované) materiály, u kterých toto není splněno a nejsou tak obsaženy, je uveden jejich zdroj, např. webová adresa, v bibliografii nebo textu práce nebo souboru README.*.

Literatura

- [1] MDN. *HTML: HyperText Markup Language* [online]. 2025 [cit. 2025-7-25]. Dostupný z: <https://developer.mozilla.org/en-US/docs/Web/HTML>.
- [2] MDN. *CSS: Cascading Style Sheets* [online]. 2025 [cit. 2025-7-25]. Dostupný z: <https://developer.mozilla.org/en-US/docs/Web/CSS>.
- [3] Frost, Brad. *Chapter 2: Atoms: Atomic Design* [online]. [cit. 2025-7-25]. Dostupný z: <https://atomicdesign.bradfrost.com/chapter-2/>.
- [4] Inc., Tailwind Labs. *Rapidly build modern websites without ever leaving your HTML: Tailwind CSS* [online]. 2025 [cit. 2025-7-25]. Dostupný z: <https://tailwindcss.com/>.
- [5] MDN. *JavaScript* [online]. 2025 [cit. 2025-7-25]. Dostupný z: <https://developer.mozilla.org/en-US/docs/Web/JavaScript>.
- [6] Microsoft. *TypeScript for JavaScript Programmers: TypeScript Documentation* [online]. 2025 [cit. 2025-7-25]. Dostupný z: <https://www.typescriptlang.org/docs/handbook/typescript-in-5-minutes.html>.
- [7] Meta. *Quick Start: React Documentation* [online]. 2025 [cit. 2025-7-25]. Dostupný z: <https://react.dev/learn>.
- [8] Foundation, OpenJs. *About Node.js®* [online]. 2025 [cit. 2025-7-25]. Dostupný z: <https://nodejs.org/en/about>.
- [9] Vercel, Inc. *Next.js Docs: Next.js Documentation* [online]. 2025 [cit. 2025-7-25]. Dostupný z: <https://nextjs.org/docs>.
- [10] IBM. *What is a REST API?: IBM Topics* [online]. 2025 [cit. 2025-7-26]. Dostupný z: <https://www.ibm.com/think/topics/rest-apis>.
- [11] Vercel, Inc. *Adding Search and Pagination: Next.js Documentation* [online]. 2025 [cit. 2025-7-28]. Dostupný z: <https://nextjs.org/learn/dashboard-app/adding-search-and-pagination>.